Say we choose `-0.5` as the median, then we have

```
(median, left, right) ← (-0.5,0), [(-1, 1),(-1.5,-2),(-1,6)],
[(1,-1), (2.5,3),(2,5)(0,5)]
```

So, at line #5 we call `constructKdTree([(-1, 1), (-1.5,-2),(-1,6)], 1)` to create root's left subtree. This in turn will partition again the sub-array, but comparing the second coordinates of each tuple: the median of `y` coordinates is 1, so we have:

```
(median, left, right) ← (-1, 1), [(-1.5,-2)], [(-1,6)]
```

And so on, the method would similarly run on the other partitions created on the initial array.

What's the running time of method `constructKdTree`? We will use `Tk(n)` to denote the running time, for a k-d tree of dimension `k`, on an array with `n` elements, let's check the method step by step: lines #1 to #3 only requires a constant amount of time, as well as line #7, which is just creating a new node. Lines #5 and #6 are recursive calls, and they will be called on sets of points with at most `n/2` elements, so we know they will take `Tk(n/2)` steps each.

Finally line #4, where we call `partition`: it's possible to find a median in linear time, and we can as well partition an array of `n` elements around a pivot with `O(n)` swaps (or create two new arrays, also with `O(n)` total assignments).

So, summing up, we have this formula for the running time: